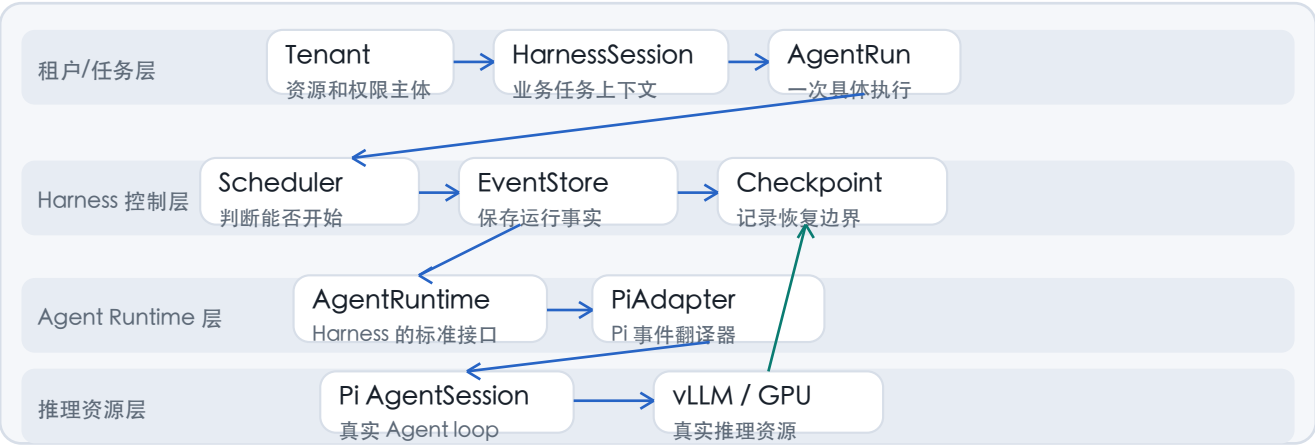


VRAM-Aware Agent Harness

概念连接图：从事件、handler、runtime、session 到资源调度

这份资料不是完整设计文档，而是一张可打印的学习地图。阅读目标是把代码里的名词和系统行为对上号，尤其是：命令如何进入 Agent，事件如何出来，handler 如何把事件变成 Harness 的状态、审计、恢复和资源控制。

当前结论	说明
Harness 不是重新写 Agent	Pi 负责 Agent loop、模型流式输出、工具协议、Pi Session 历史。
Harness 是控制层	Harness 负责 AgentRun 生命周期、事件持久化、工具治理、checkpoint、恢复、资源观测和调度。
Adapter 是边界	PiAdapter 把 Harness 命令翻译成 Pi SDK 调用，再把 Pi 原始事件翻译成 Harness RuntimeEvent。
事件是系统主线	后续 EventStore、RunState、Checkpoint、Scheduler、ResourceMonitor 都是通过事件接入。



1. 先用一句话抓住系统

Harness 做的事情是：把一次 Agent 执行拆成一串标准事件，然后让不同 handler 分别处理这些事件。

所以你现在写的 runtime/adaptor 不是业务完整体，而是事件入口：Agent 开始、输出文字、调用工具、工具完成、被中断、失败、成功结束，这些事实都必须从底层 Agent Runtime 流入 Harness。



问题	正确理解
emit 发出的是什么？	emit 发出的是 Harness 自己定义的 RuntimeEvent，不是 Pi 原始事件。
handler 是什么？	handler 是事件发生后要执行的函数，比如写数据库、更新状态、推送 UI。
subscribe 是什么？	subscribe 是把 handler 注册到某个 runId 下面。事件到了，emit 会找这个 runId 的 handlers 并调用。
为什么不直接调用 handler？	因为未来同一个事件要被多个模块处理；订阅机制让 PiAdapter 不依赖具体上层服务。

2. 对象关系：不要把这些 ID 混在一起

名字	它是谁	它解决的问题	例子
tenantId	资源和权限主体	公平性、配额、审计	tenant-a
harnessSessionId	Harness 业务会话	把一个用户任务的上下文聚合起来	session-fix-readme
runId	一次具体执行	记录一次 start/resume/interrupt 的运行事实	run-001
runtimeSessionRef	底层 Runtime 会话引用	恢复时重新打开 Pi 的会话历史	Pi sessionFile
checkpointId	Harness 保存的恢复点	知道从哪个安全边界恢复	checkpoint-001

核心关系

一个 harnessSession 可以有多个 AgentRun。比如第一次执行是 run-001，中断后恢复可能是 run-002，失败后重试可能是 run-003。它们都属于同一个业务任务，但每次执行都应该有独立的运行事实。

AgentRun 执行时会使用底层 Pi AgentSession。Pi AgentSession 的持久化引用就是 runtimeSessionRef。Harness 不直接理解 Pi 的内部历史结构，但 Harness 必须保存这个引用，以便 resume 时交还给 Pi。

简化记忆：harnessSessionId 管业务任务；runId 管一次执行；runtimeSessionRef 管底层 Pi 历史；checkpointId 管恢复边界。

3. AgentRuntime 和 PiAdapter 到底在做什么

AgentRuntime 是 Harness 对底层 Agent 执行引擎提出的标准接口。它不是 Pi，也不是某个具体 Agent，而是一份契约：任何底层实现都必须支持 start、resume、interrupt、subscribe。

PiAdapter implements AgentRuntime，意思是：当前我们用 Pi 来满足这份契约。将来如果替换成别的 SDK，上层 Harness 仍然只面对 AgentRuntime。

```
interface AgentRuntime { start(request): Promise<void>; resume(request): Promise<void>; interrupt(runId): Promise<void>; subscribe(runId, handler): () => void; }
```

方法	从事件角度看它的意义
start(request)	命令：启动一次真实 Agent 执行。之后应该产生 agent_started、text_delta、tool_started 等事件。
resume(request)	命令：从 checkpoint 恢复。之后应该产生 agent_resumed，然后继续产生后续事件。
interrupt(runId)	命令：请求中断正在运行的 Agent。真正成功中断后，PiAdapter 应 emit agent_interrupted。
subscribe(runId, handler)	注册：把 handler 放进 runId 对应的事件监听集合。

PiAdapter 当前职责边界

它只做翻译和生命周期连接：Harness 命令 -> Pi SDK 调用；Pi 原始事件 -> Harness RuntimeEvent。它不应该直接做 tenant 配额、GPU 策略、数据库 schema、业务重试策略。那些属于上层服务。

4. start / interrupt / resume 的事件语义



动作	它是命令还是事实	应该由谁发出/处理
start	命令	RunService/Scheduler 决定可以开始后，调用 AgentRuntime.start。
agent_started	事实事件	PiAdapter 在收到 Pi agent_start 后 emit，并带上 runtimeSessionRef。
interrupt	命令	外部请求中断时调用 AgentRuntime.interrupt(runId)。
agent_interrupted	事实事件	PiAdapter 根据 Pi agent_end + stopReason=aborted 发出。
resume	命令	Recovery/Checkpoint 服务拿 checkpoint 调用 AgentRuntime.resume。
agent_resumed	事实事件	PiAdapter 真的打开底层 session 并继续运行后发出。

注意：Pi 自己可能只发 generic 的 agent_start。Harness 要区分 start 和 resume，因此 PiAdapter 需要根据自己当前执行的是 start 还是 resume，把它翻译成 agent_started 或 agent_resumed。

5. 事件如何驱动后续 Harness 服务

收到的 RuntimeEvent	典型 handler	后续系统状态
agent_started	EventStoreHandler / RunStateHandler / CheckpointHandler	写入 run 起点，保存 runtimeSessionRef，状态变 running。
text_delta	EventStoreHandler / UIStreamHandler	保存输出片段，实时显示给用户。
tool_started	ToolGovernanceHandler / AuditHandler	记录工具调用开始，未来可在这里做权限、超时、并发限制。
tool_completed	EventStoreHandler / CheckpointHandler / MetricsHandler	保存工具结果，判断是否形成安全恢复边界。
agent_interrupted	RunStateHandler / RecoveryHandler	状态变 interrupted，等待恢复或人工处理。
agent_failed	RunStateHandler / RecoveryHandler	状态变 failed，判断是否可自动恢复。
agent_completed	RunStateHandler / CleanupHandler	状态变 completed，释放活跃运行资源。

这也是为什么我们先写 runtime，而不是先写 GPU

调度。没有标准事件，上层服务就没有可信输入。先把事件入口打通，后面所有控制逻辑才有事实依据。

6. GPU/资源调度在架构里的位置

Harness 会感知 GPU 资源，但不是 PiAdapter 直接做。正确位置通常是 ResourceMonitor + Scheduler + PolicyEngine。PiAdapter 只负责让 Agent 跑起来，并把运行事实吐出来。

模块	它看什么	它决定什么
ResourceMonitor	GPU 显存、vLLM 队列、token 吞吐、请求延迟	当前资源事实是什么。
Scheduler	tenant 配额、run 优先级、资源水位、等待时间	这个 run 现在能不能 start/resume。
PolicyEngine	任务语义、历史效果、SLO、资源趋势	更高级的准入、优先级、预算策略。
PiAdapter	Pi session 和 Pi events	不做资源决策，只执行被允许的 run。

固定策略和 Agentic 策略的关系

第一阶段先用确定性规则：比如显存低于阈值时不启动新 run，tenant 超额时排队。后续可以加入 Scheduling Agent，但必须先 Shadow Mode、离线评估，再在 hard guardrails 内参与真实调度。

面试表达：Agentic Scheduling 不是让模型随便控制 GPU，而是在可审计的硬约束内，让它基于资源趋势、任务语义和历史结果提出调度建议。

7. 当前代码对应的学习检查表

你应该能解释的问题	回答要点
为什么要有 AgentRuntime ?	让 Harness 不依赖 Pi 的具体 API，上层只依赖 start/resume/interrupt/subscribe。
PiAdapter 里 emit 的是什么？	Harness RuntimeEvent，不是 Pi 原始事件。
subscribe 为什么按 runId 存 handler？	避免不同 run 的事件串线；一个 run 可以有多个 handler。
interrupt 为什么不是直接 emit agent_interrupted？	interrupt 是命令；只有 Pi 确认 abort 后，agent_interrupted 才是事实。
runtimeSessionRef 有什么用？	恢复钥匙。Checkpoint 保存它，resume 用它重新打开底层 Pi Session。
为什么现在 resume 可以暂缓？	因为真实 resume 需要 checkpoint、事件序号、工具副作用语义和继续输入协议。先把 start/interrupt 事件链打稳。

最短主线

命令进入 AgentRuntime；PiAdapter 调 Pi；Pi 产生原始事件；PiAdapter 翻译成 RuntimeEvent；emit 找到订阅的 handlers；handlers 更新数据库、状态、checkpoint、资源统计和 UI。